Neuromorphic ASR
Transducer Models on Memristor Hardware

Azim Javed
azim.javed@ml.rwth-aachen.de

August 25, 2026

Advisor: Benedikt Hilmes and Simon Berger
Supervisor: Prof. Ralf Schlüter
Human Language Technology and Pattern Recognition
Computer Science Department, RWTH Aachen University

Outline

Outline

Outline

Related Work

Prerequisites

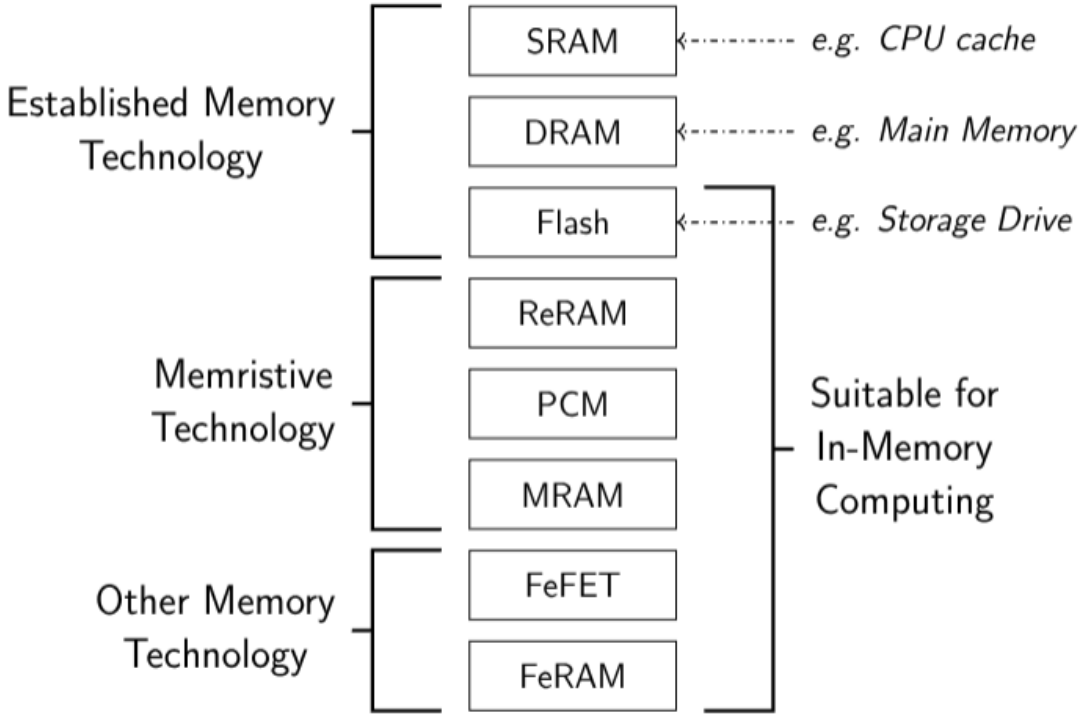
Motivation

Experiments

Future Work

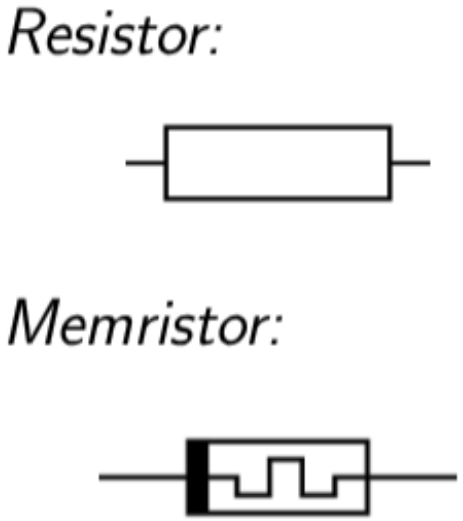
In-Memory Computing [?]

- Various technologies exist, with varying tradeoffs:
  - Chip complexity and density
  - Energy consumption
  - Speed and reliability
- This talk focuses on memristive hardware, in particular resistive RAM (ReRAM):
  - Key Benefits: low energy consumption, non volatility



Overview[?]

- A memristor is an analog computing device with a programmable resistance states.
- Enables energy-efficient implementation of neural networks.
  - Vector-matrix-multiplication (VMM) in $\mathcal{O}(1)$
- Limitations:
  - High analog inaccuracies, and thus need specific restrictions on mathematical operations.
  - Production is still in early stages, experiments are on simulated hardware [**?**].



Analog In Memory Computing

- Multiplication computation done via Ohm's Law:

$$V = IR \tag{1}$$

- Voltage V, current I and resistance R.
- Possible to rewrite resistance as conductance G:

$$I = GV \quad \text{with} \quad G = \frac{1}{R} \tag{2}$$

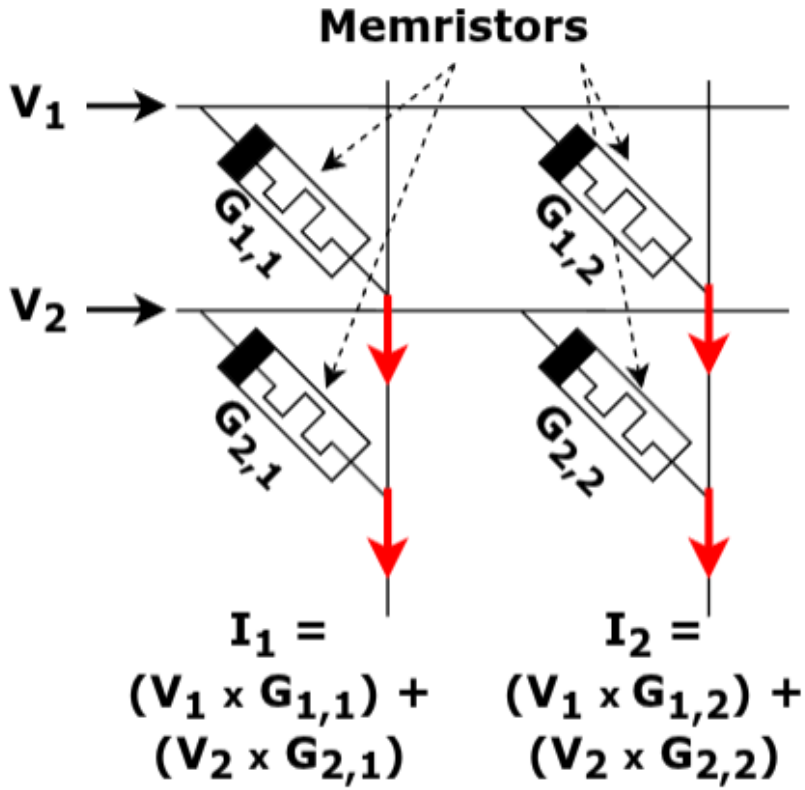
- Addition done via Kirchhoff's Law for any connected node in a circuit:

$$\sum_{n=0}^{N} I_n = 0 \;\rightarrow\; I_0 = -\sum_{n=1}^{N} I_n \tag{3}$$

- Currents can be trivially added by simply connecting electric lines.

Analog In Memory Computing (2)

- Memristors as connecting element between vertical and horizontal circuit lines
- Each memristor is an element of a constant matrix
- Column sum via simple line connection

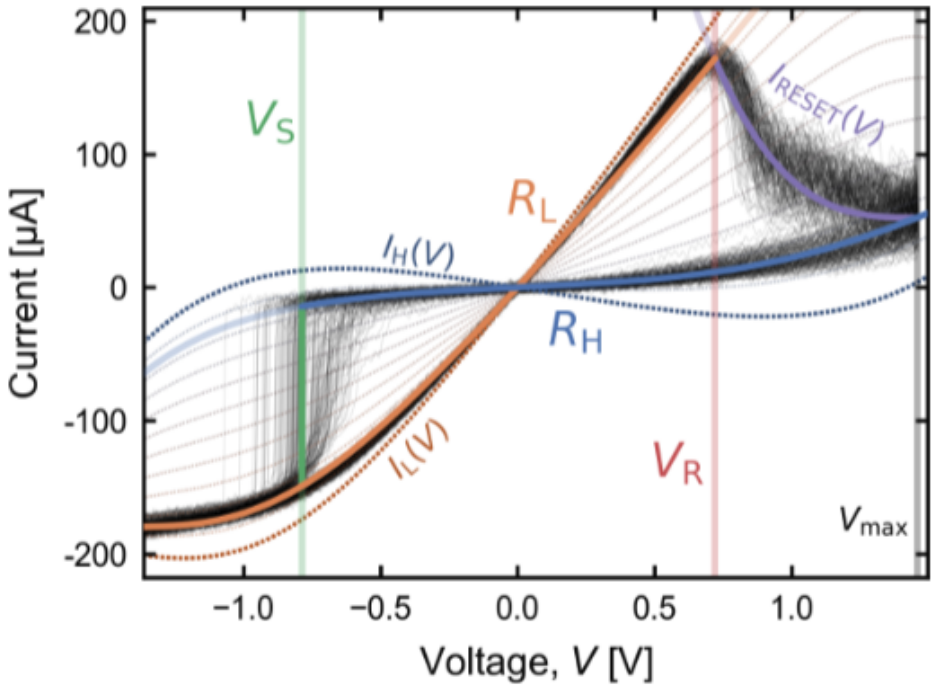


Could show the resistors here instead of a memristor

The Memristor

Key Idea: Program weights by setting the resistance.

- Apply $-V > -V_S \rightarrow$ set to low resistance
- Apply $V > V_R \rightarrow$ set to high resistance
- Intermediate states possible
- State is physically stored (non-volatile memory)



Limitations

Programming Limitations

- Resulting resistance state varies with each programming cycle
- Non-linear relation of current and voltage

Cannot exactly model weights

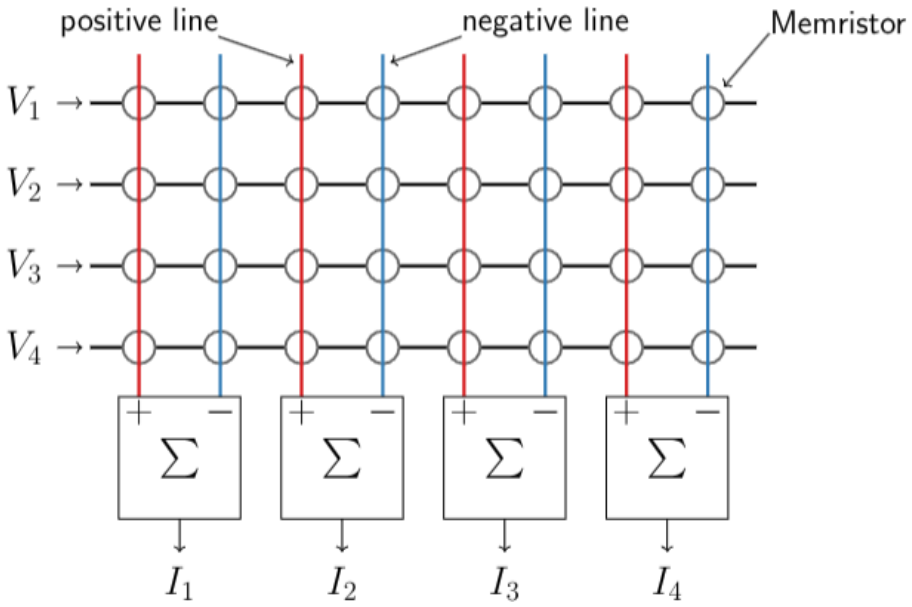
- Only positive, non-zero weights possible
- Current leakage prohibits a true zero

(Paired) Memristor Array

- Use positive and negative bitlines to create memristor pairs
- Each row-wise pair of memristors represents a weight

→ Differential pair allows for negative and zero weight values:

$$I_j = \sum_{i=1}^{N} V_i G_{i,j}^{+} - \sum_{i=1}^{N} V_i G_{i,j}^{-}$$



Problems:

- Cannot set paired memristors to identical conductances

→ Difference close to zero, but not guaranteed

Memristor Neural Layers

- Tile the weight matrix:
  - Tile weight matrix with arbitrary dimensions into smaller fixed-size arrays.
  - E.g., 512×512 weight matrix $\rightarrow 16$ crossbar arrays of size 128×128
- Slice weights into memristor states:
  - Binary setting (high vs. low resistance)
  - Stacking of memristors for each magnitude bit (k-bit weight requires $k - 1$ arrays)
- Pair positive $(+)$ and negative $(-)$ bitlines.

Memristor Neural Layers

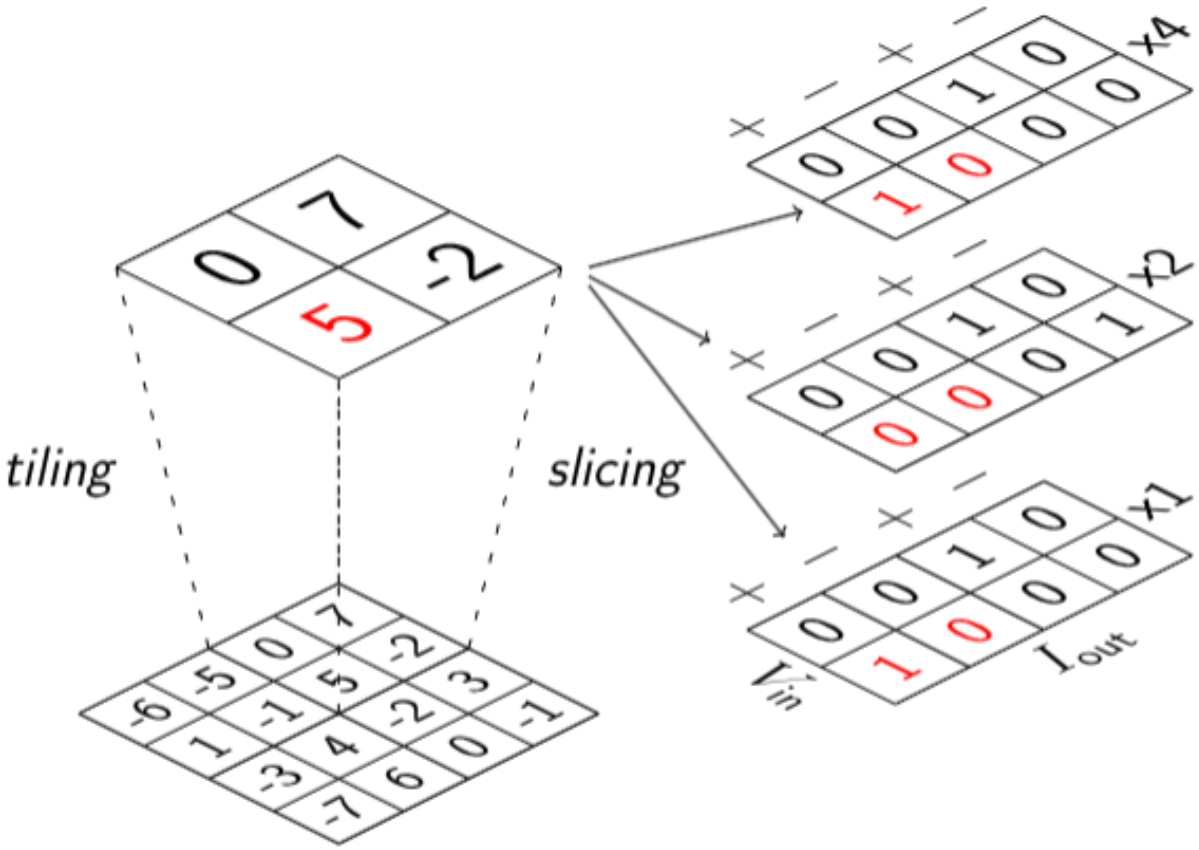
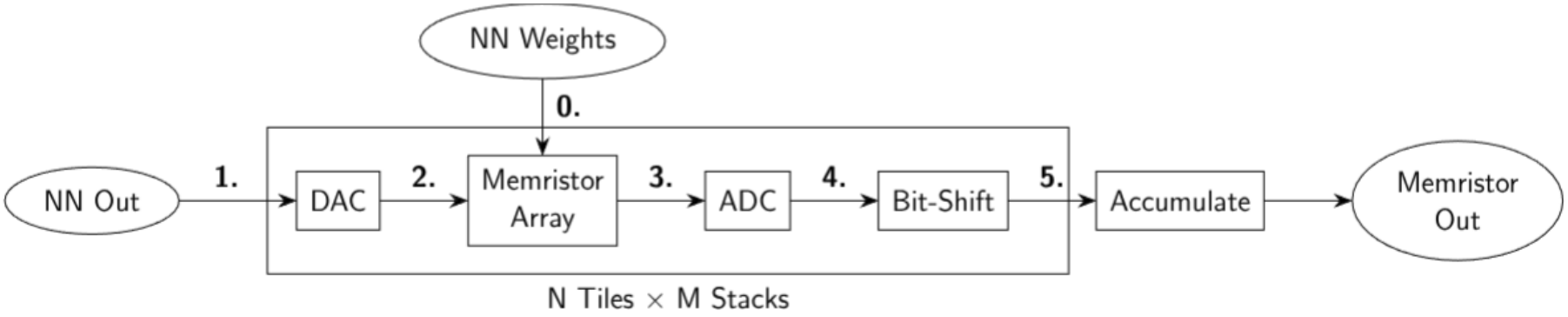


Figure: Mapping a weight matrix on memristor arrays [Rossenbach & Hilmes+ 25].

Execution of Memristor Layer

0. Program memristors by applying either a large voltage
1. Normalize input and quantize with the (Digital-Analog Converter) DAC settings
2. Apply input voltage to the cell via simulation function
3. Read current from crossbars
4. Apply ADC (Analog-Digital Converter) conversion back into digital domain
5. Bit-shift and accumulate result



Quantization

- The number of memristors required is proportional to the precision of the weights.
  - ▷ Need to reduce the precision of the weights → quantization.
- Quantization-Aware Training (QAT) peforms better than Post-training Quantization (PTQ), but requires more training time [**?**].

  → High performance gap at low precision
- Symmetric per-tensor QAT, zero-point fixed to 0.

Table: WER (%) on TED-LIUMv2 dev, Conformer CTC, 4-gram LM [Rossenbach & Hilmes+ 25]

Weight bits	FP32	8	6	5	4	3	2
PTQ	7.2	7.2	7.9	8.9	11.4	30.7	–
QAT		7.3	7.7	7.4	7.8	8.3	22.1

Also need to tell the requirement for observers during training and the memristor programming with observers, scale, ADC, DAC missing. Perhaps need another quantization slide.

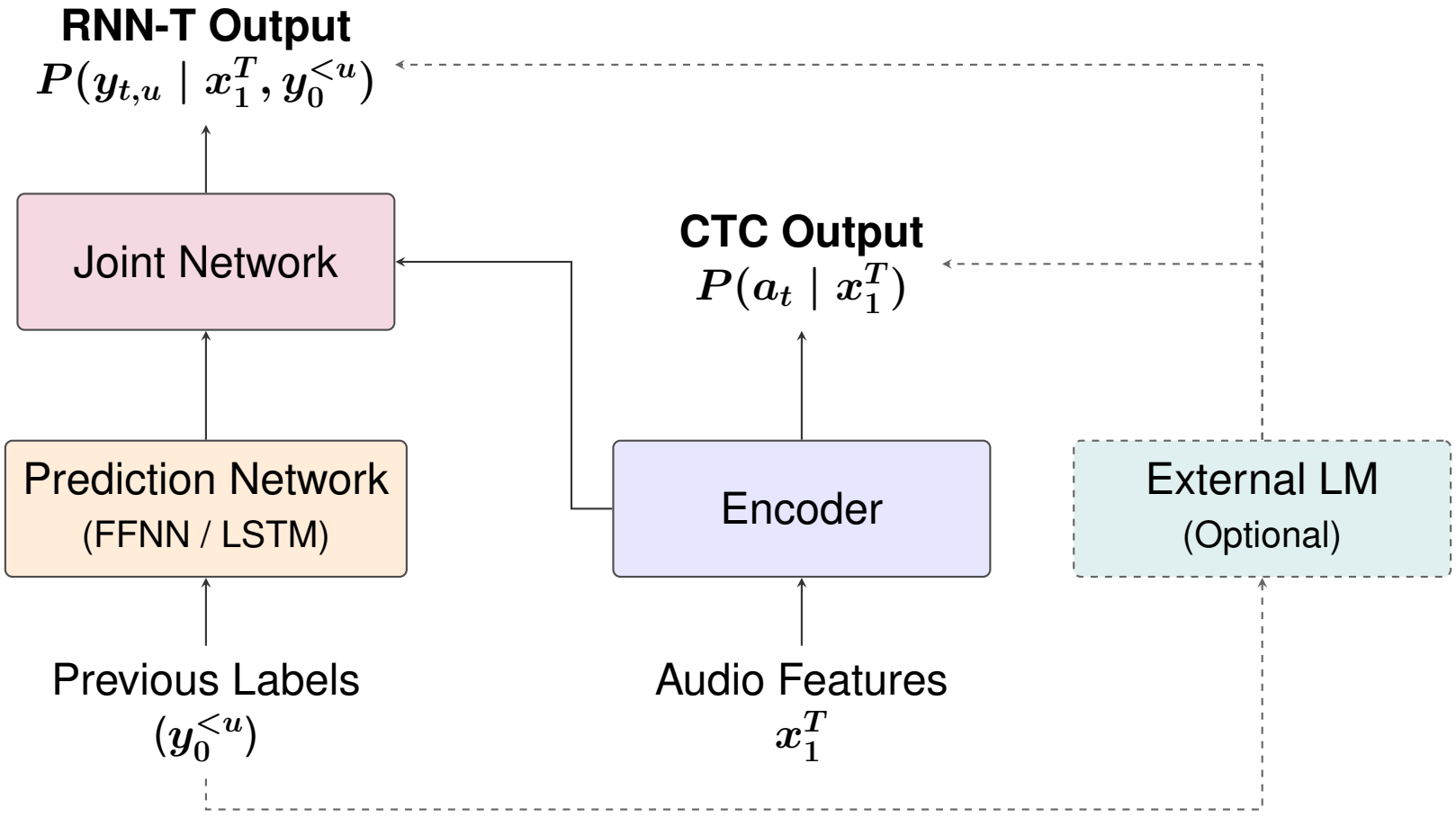
Outline

Motivation

- CTC-based Conformer shown to run on (simulated) memristor hardware [Rossenbach & Hilmes+ 25].
- How do transducer models, with their added complexity compared to CTC, behave on device?
  - Autoregressive prediction network; internal language model (ILM).
  - Joint network evaluated repeatedly per emitted label.
- Performance degradation across search paradigms -
  - Lexicon-free vs. tree search.
  - ILM and prior scaling - How does device noise change the optimal LM weighting?

Motivation

- Performance impact of deploying components of the model on the device?
  - Encoder only vs. encoder + prediction network vs. full model (encoder + prediction network + joint network)
- Tradeoff: more offloading $\Rightarrow$ more energy efficiency, but also more noise.



Outline

Experimental Setup

- Data: LibriSpeech [Panayotov & Chen$^+$ 15] (960 h), BPE subword targets (128 merges $\rightarrow$ 184 labels).
- Features: 80 log-mel filters, 10 ms frame shift; SpecAugment during training.
- Encoder: 12-layer Conformer [Gulati & Qin$^+$ 20], 512-dim.
- Model types:
  1. CTC [Graves & Fernández$^+$ 06]: encoder only.
  2. FFNN-transducer: prediction network is a 2-layer feed-forward net (640-dim) over the previous-label embedding.
  3. Full-context transducer [Graves 12]: LSTM prediction network.
- Quantization: QAT, per-tensor symmetric, 8 and 4-bit weights, 8-bit activations; zero-point 0.
- Stages: (1) Full precision baseline $\rightarrow$ (2) QAT (fake quantization) $\rightarrow$ (3) memristor simulation (3-5 runs).

CTC Experiments

Table: WER (%) on LibriSpeech dev-other across CTC configurations

(a) Baseline

	Lex-Free		Tree
Model	**No LM**	**LSTM**	**4-gram**
CTC	7.1	5.0	5.2

(b) QAT

	Lex-Free		Tree
Bits	**No LM**	**LSTM**	**4-gram**
8	7.3	5.2	5.3
4	8.3	5.4	5.7

(c) Memristor Simulation

	Lex-Free		Tree
Bits	**No LM**	**LSTM**	**4-gram**
8	13.53 ± 0.37	–	6.67 ± 0.09
4	11.85 ± 0.18	–	6.77 ± 0.12

FFNN Transducer Experiments

Table: WER (%) on LibriSpeech dev-other across FFNN-Transducer configurations

(a) Baseline

	Lex-Free		Tree
Model	**No LM**	**LSTM**	**4-gram**
FFNN-Transducer	–	–	–

(b) QAT

Quantized				Lex-Free		Tree
Encoder	**Prediction**	**Joiner**	**Bits**	**No LM**	**LSTM**	**4-gram**
✓	–	–	8	–	–	–
✓	✓	–	8	–	–	–
✓	✓	✓	8	–	–	–
✓	–	–	4	–	–	–
✓	✓	–	4	–	–	–
✓	✓	✓	4	–	–	–

(c) Memristor Simulation

Quantized				Lex-Free		Tree
Encoder	**Prediction**	**Joiner**	**Bits**	**No LM**	**LSTM**	**4-gram**
✓	–	–	8	–	–	–
✓	✓	–	8	–	–	–
✓	✓	✓	8	–	–	–
✓	–	–	4	–	–	–
✓	✓	–	4	–	–	–
✓	✓	✓	4	–	–	–

FFNN Transducer: Fine-Tuning vs. QAT from Scratch

Table: WER (%) on LibriSpeech dev-other: Checkpoint fine-tuning vs. training from scratch

(a) QAT

		Lex-Free		Tree
Training Scheme	**Bits**	**No LM**	**LSTM**	**4-gram**
Fine-tuned	8	–	–	–
From Scratch	8	–	–	–
Fine-tuned	4	–	–	–
From Scratch	4	–	–	–

(b) Memristor Simulation

		Lex-Free		Tree
Training Scheme	**Bits**	**No LM**	**LSTM**	**4-gram**
Fine-tuned	8	–	–	–
From Scratch	8	–	–	–
Fine-tuned	4	–	–	–
From Scratch	4	–	–	–

Full Context Transducer Experiments

Table: WER (%) on LibriSpeech dev-other across Full-Context Transducer configurations

(a) Baseline

	Lex-Free		Tree
Model	**No LM**	**LSTM**	**4-gram**
Full-Context Transducer	–	–	–

(b) QAT

Quantized				Lex-Free		Tree
Encoder	**Prediction**	**Joiner**	**Bits**	**No LM**	**LSTM**	**4-gram**
✓	–	–	8	–	–	–
✓	✓	–	8	–	–	–
✓	✓	✓	8	–	–	–
✓	–	–	4	–	–	–
✓	✓	–	4	–	–	–
✓	✓	✓	4	–	–	–

(c) Memristor Simulation

Quantized				Lex-Free		Tree
Encoder	**Prediction**	**Joiner**	**Bits**	**No LM**	**LSTM**	**4-gram**
✓	–	–	8	–	–	–
✓	✓	–	8	–	–	–
✓	✓	✓	8	–	–	–
✓	–	–	4	–	–	–
✓	✓	–	4	–	–	–
✓	✓	✓	4	–	–	–

Optimal Parameters during QAT and Memristor Simulation

Table: Impact of quantization and device noise on optimal LM scales (LibriSpeech dev-other)

(a) Baseline (FP32)

ILM Scale	ELM Scale	WER (%)
0.1	0.2	–
0.2	0.3	–
0.2	**0.4**	**–**
0.3	0.4	–
0.4	0.5	–

(b) QAT

ILM Scale	ELM Scale	WER (%)
0.1	0.2	–
0.2	0.3	–
0.2	0.4	–
0.3	**0.4**	**–**
0.4	0.5	–

(c) Memristor Simulation

ILM Scale	ELM Scale	WER (%)
0.1	0.2	–
0.2	0.3	–
0.2	0.4	–
0.3	**0.4**	**–**
0.4	0.5	–

ILM: Internal Language Model, **ELM**: External Language Model

RTF - Memristor Simulation

Table: Real-Time Factor (RTF) on simulated memristor hardware (8-bit quantization, dev-other)

(a) Standard Evaluation

Model	**RTF (Tree 4-Gram)**
CTC	–
FFNN-Transducer	–
Full-Context Transducer	–

(b) Optimizations under relaxed simulation assumptions

Model	**Caching Scheme**	**RTF (Tree 4-Gram)**
FFNN-Transducer	Prediction network output cached	–
Full-Context Transducer	LSTM input layer cached	–

RTF: Real-Time Factor (RTF < 1.0 indicates real-time processing).

Outline

Related Work

Prerequisites

Motivation

Experiments

Future Work

Future Work

- Error Analysis: Which error types does device noise introduce or amplify?
  - Substitutions, insertions, deletions;
  - Worse degradation across certain search paradigms (lexicon-free)
- Countermeasures:
  - Denoising (e.g. MLP),
  - Tuning search parameters, LM parameters, softmax temperature etc.
- Fine-tuning of the (quantized) model
  - Introduction of device-like noise during QAT training.
- Generalization: Test models with different datasets and conditions.

Thank you for your attention!

Reference

- A. Graves, S. Fernández, F. Gomez, J. Schmidhuber.
  Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks.
  In *Proc. ICML*, pp. 369–376, Pittsburgh, PA, June 2006.
- A. Graves.
  Sequence Transduction with Recurrent Neural Networks.
  In *Proc. ICML*, Edinburgh, Scotland, June 2012.
  Workshop on Representation Learning, arXiv:1211.3711.
- A. Gulati, J. Qin, C.-C. Chiu, N. Parmar, Y. Zhang, J. Yu, W. Han, S. Wang, Z. Zhang, Y. Wu, R. Pang.
  Conformer: Convolution-Augmented Transformer for Speech Recognition.
  In *Proc. Interspeech*, pp. 5036–5040, Shanghai, China, Oct. 2020.
- V. Panayotov, G. Chen, D. Povey, S. Khudanpur.
  Librispeech: an ASR Corpus Based on Public Domain Audio Books.
  In *Proc. IEEE ICASSP*, pp. 5206–5210, Queensland, Australia, April 2015.

N. Rossenbach, B. Hilmes, L. Brackmann, M. Gunz, R. Schlüter.
Running Conventional Automatic Speech Recognition on Memristor Hardware: A Simulated Approach.
In *Proc. Interspeech*, Rotterdam, The Netherlands, Aug. 2025.
arXiv:2505.24721.

Appendix: Device VMM Operation Statistics

operation	average	std-dev	min	max
1.0×1	1.016	0.063	0.707	1.27
0.1×1	$0.989 \cdot 10^{-1}$	$0.062 \cdot 10^{-1}$	$0.688 \cdot 10^{-1}$	$1.24 \cdot 10^{-1}$
0.01×1	$0.985 \cdot 10^{-2}$	$0.069 \cdot 10^{-2}$	$0.721 \cdot 10^{-2}$	$1.20 \cdot 10^{-2}$
1.0×0	$8.52 \cdot 10^{-4}$	$4.75 \cdot 10^{-2}$	-0.2751	0.2455
1.0×0.5	0.476	0.094	0.109	1.06

Table: Hardware measurement statistics for simulated memristor VMM operations [Rossenbach & Hilmes+ 25].